# STIJK2124 Distributed Computing — Campus Ride

## Part 6: Wireshark Network Analysis — OSI Layer Observations

| Ex. | Layer(s) | What Was Observed | Key Packet Detail | DS Concept |
|---|---|---|---|---|
| **6.1** | L4 Transport | TCP 3-way handshake | SYN → SYN-ACK → ACK flags; seq numbers | Reliable connection setup before data exchange |
| **6.2** | L1–L7 (all) | Full protocol stack expanded in one packet | Frame/Ethernet/IP/TCP/HTTP headers nested | Encapsulation & layered abstraction |
| **6.3** | L6 Presentation | HTTP POST with JSON body visible | Content-Type: application/json; decoded payload | Data serialisation across service boundaries |
| **6.4** | L4 Transport | TCP 4-way connection termination | FIN-ACK → ACK → FIN-ACK → ACK sequence | Graceful resource release / connection draining |
| **6.5** | L2–L3 Network | Docker container-to-container IP traffic | 172.18.x.x private IPs; virtual veth MACs | Virtual networking & DNS service discovery |
| **6.6** | L4–L7 | TLS handshake then encrypted application data | Client Hello → Server Hello → {Encrypted} | End-to-end encryption for distributed auth |

### Exercise 6.1 — TCP Three-Way Handshake | Layer 4: Transport

Observed

Capture filter host 127.0.0.1 and port 8000 against curl http://127.0.0.1:8000/health produced three TCP control packets before any HTTP data: (1) [SYN] from client ephemeral port to server port 8000 with initial sequence number (ISN); (2) [SYN, ACK] from server acknowledging ISN+1 and announcing its own ISN; (3) [ACK] from client completing the exchange. No application payload is present in any of these packets — they are pure Layer 4 headers containing source/destination ports, sequence numbers, flags, and window size.

**DS Concept — Reliable Connection Establishment:**

The handshake ensures both endpoints are reachable and agree on sequence numbers before a single byte of REST API data is sent. In Campus Ride's multi-container architecture, this prevents stale or misdirected packets from corrupting FastAPI responses. Window size negotiation during the handshake also enables TCP flow control — essential when the backend serves many concurrent clients under Kubernetes load balancing.

### Exercise 6.2- HTTP GET with Full Layer Expansion

**Observed:**

Clicking the HTTP packet from curl http://127.0.0.1:8000/api/v1/students/ and expanding all layers revealed: Frame (L1) — captured length ~312 bytes, timestamp, loopback interface. Ethernet II (L2) — null MACs (loopback), EtherType 0x0800. IPv4 (L3) — src/dst 127.0.0.1, Protocol TCP, TTL 64. TCP (L4) — src ephemeral port, dst 8000, flags [PSH, ACK]. HTTP (L7) — GET /api/v1/students/ HTTP/1.1, Host: 127.0.0.1:8000, User-Agent: curl/8.x. Layers 5–6 are implicit: TCP maintains the session; plain HTTP requires no encoding transformation.

**DS Concept — Layered Abstraction & Encapsulation:**

Each layer adds its own header and treats layers below as a black box. The FastAPI Python code only handles the Layer 7 HTTP request; it is unaware of TCP sequence numbers or Ethernet MACs. This separation allows Campus Ride to add TLS (Exercise 6.6) between L4 and L7 with no change to application logic — a key principle enabling independent evolution of distributed system components.

### Exercise 6.3 – HTTP POSR with JSON Payload

**Observed:**

Display filter http.request.method == "POST" isolated the curl POST packet. Layer 7 header: POST /api/v1/students/ HTTP/1.1, Content-Type: application/json, Content-Length matching body size. Wireshark decoded the Layer 6 payload as structured JSON: {"name":"Ali","matric_id":"S001","email":"ali@uni.edu","lat":6.45,"lng":100.50}. The TCP stream (L4) continued from the previous GET with advanced sequence numbers and [PSH, ACK] flags.

**DS Concept — Data Serialisation Across Service Boundaries:**

JSON is the Presentation layer encoding that allows a TypeScript frontend object to become identical Python data inside FastAPI without manual byte manipulation. This language-agnostic serialisation is fundamental to microservice communication in Campus Ride. High-throughput distributed systems extend this concept to binary formats (Protocol Buffers, MessagePack) to reduce the verbosity overhead visible in this capture.

### Exercise 6.4 TCP Four-Way Termination

**Observed:**

Post-response, four TCP control packets terminated the connection: [FIN, ACK] from client signalling no more data; [ACK] from server (half-close — server may still send); [FIN, ACK] from server closing its side; [ACK] from client. The client then entered TIME_WAIT for 2×MSL before the ephemeral port was reclaimed. Sequence numbers incremented across all four packets, confirming stream integrity.

**DS Concept — Graceful Resource Release:**

Improper teardown in a distributed system causes port and memory exhaustion. The four-way FIN ensures both sides confirm completion before kernel resources are freed. In Campus Ride's Kubernetes deployment, terminationGracePeriodSeconds and connection draining operate at this same layer — pods wait for all in-flight TCP connections to complete FIN sequences before shutdown, preventing dropped requests during scaling events.

### Exercise 6.5 – Docker Cointainer Traffic

**Observed:**

On the docker0 / vEthernet interface with filter port 5432 or port 8000, all inter-container packets carried private 172.18.x.x IP addresses — e.g., campus-backend at 172.18.0.3 querying campus-postgres at 172.18.0.2. Layer 2 Ethernet headers showed virtual MAC addresses assigned by Docker's veth pairs. docker network inspect campus-ride_default confirmed the 172.18.0.0/16 subnet and 172.18.0.1 gateway (the docker0 bridge itself).

**DS Concept — Virtual Networking & DNS-Based Service Discovery:**

Docker's bridge network is a software-defined Layer 2 LAN implemented in the Linux kernel. Its embedded DNS resolver maps container hostnames (e.g., 'postgres') to current container IPs, so the backend's connection string postgres://postgres:5432 remains valid even if the database container restarts with a new IP. This DNS-based service discovery — observable here at Layers 2–3 — is the foundation of Kubernetes CoreDNS, which extends the same pattern cluster-wide across multiple nodes.